Test using Spock

A testing framework embraces simplicity, productivity and art!

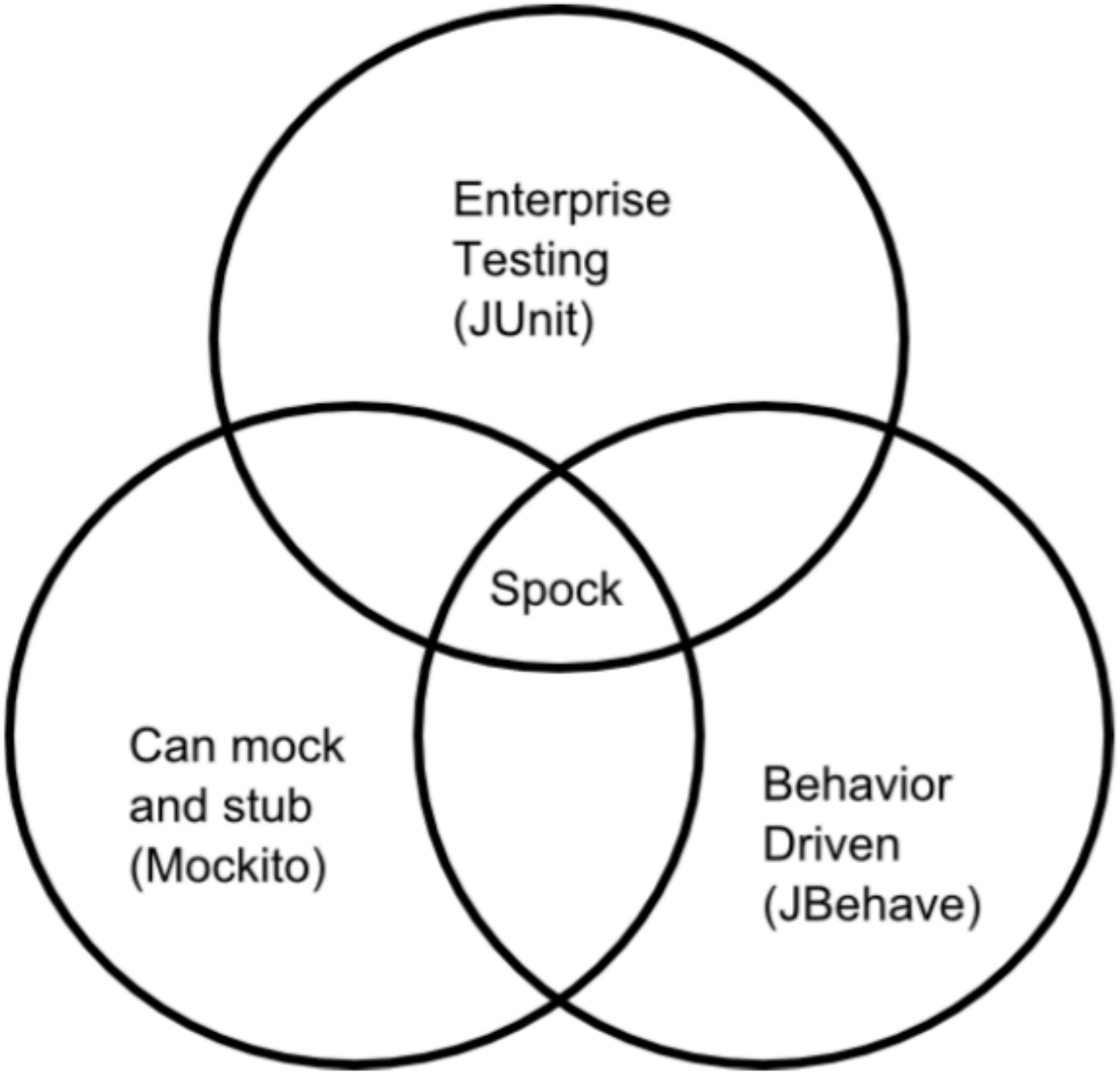
Java Unit Testing history

- JUnit - first xUnit for Java - 2000
- TestNG - Java 5 leveraged in tests - 2004
  - with some unique (at the time) features
- JUnit 4 - Java 5 support - 2006
  - catching up TestNG features over years
  - de facto standard, steady evolution rather than revolution
- Spock - revamped test creation - 2009
  - power of Groovy under the hood
- JUnit 5 - redesigned and rewritten from scratch - 2017

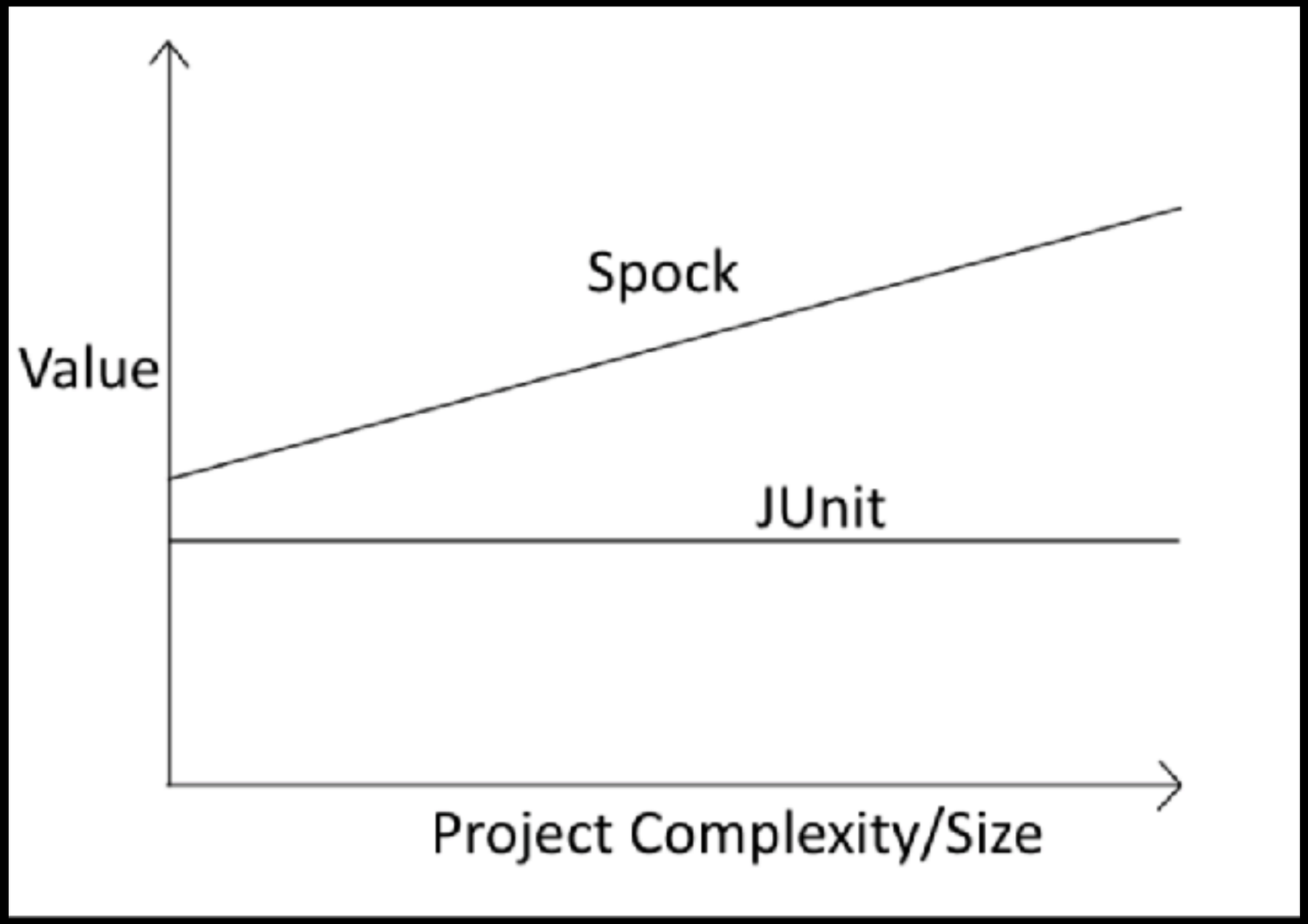
What is Spock

- Spock is a testing and specification framework for Java and Groovy applications. What makes it stand out from the crowd is its beautiful and highly expressive specification language. Thanks to its JUnit runner, Spock is compatible with most IDEs, build tools, and continuous integration servers. Spock is inspired from JUnit, RSpec, jMock, Mockito, Groovy, Scala, Vulcans, and other fascinating life forms.—http://spockframework.org/

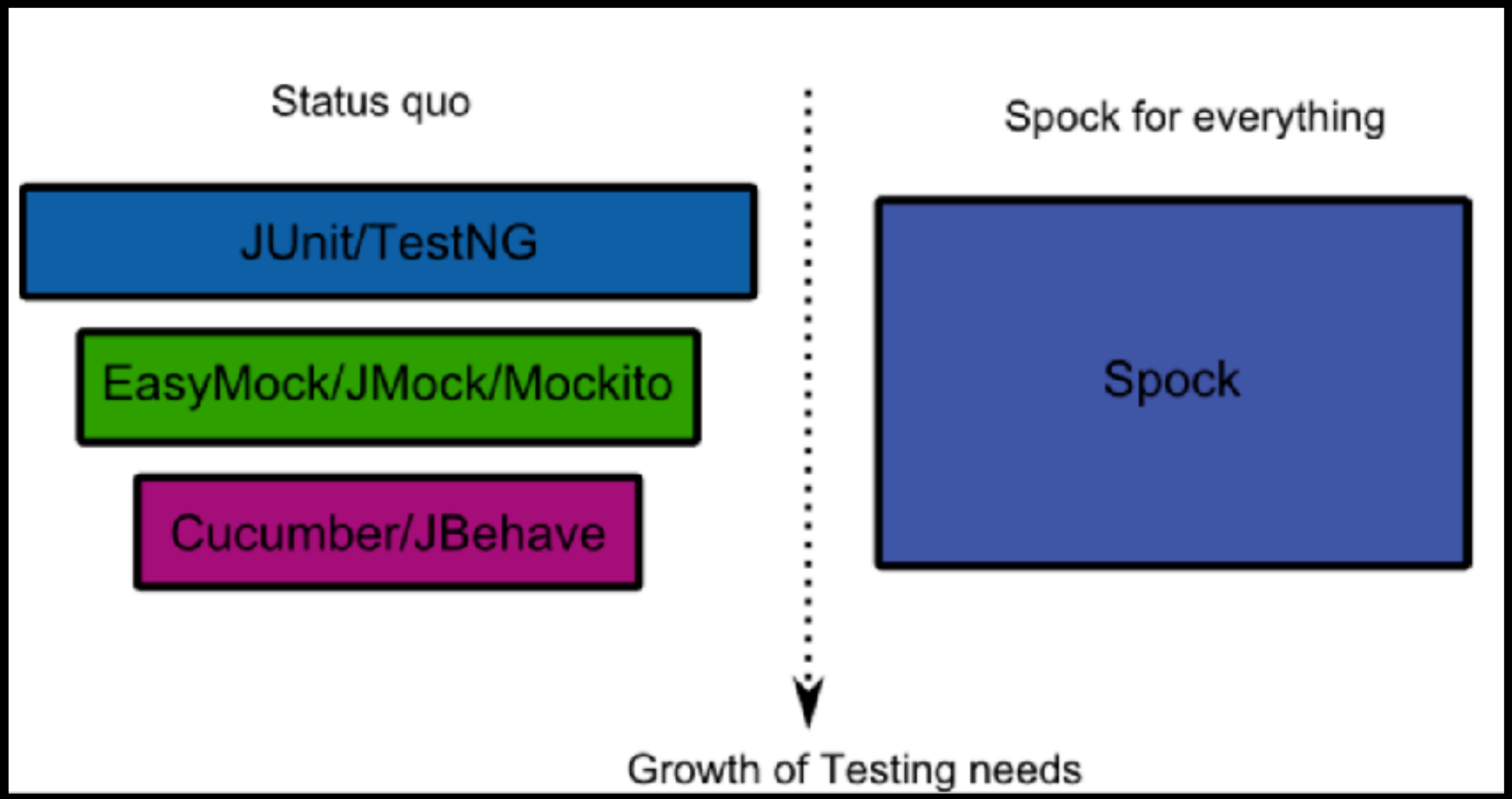
Why Spock


Enterprise
Testing
(JUnit)
Spock
Can mock
and stub
(Mockito)
Behavior
Driven
(JBehave)

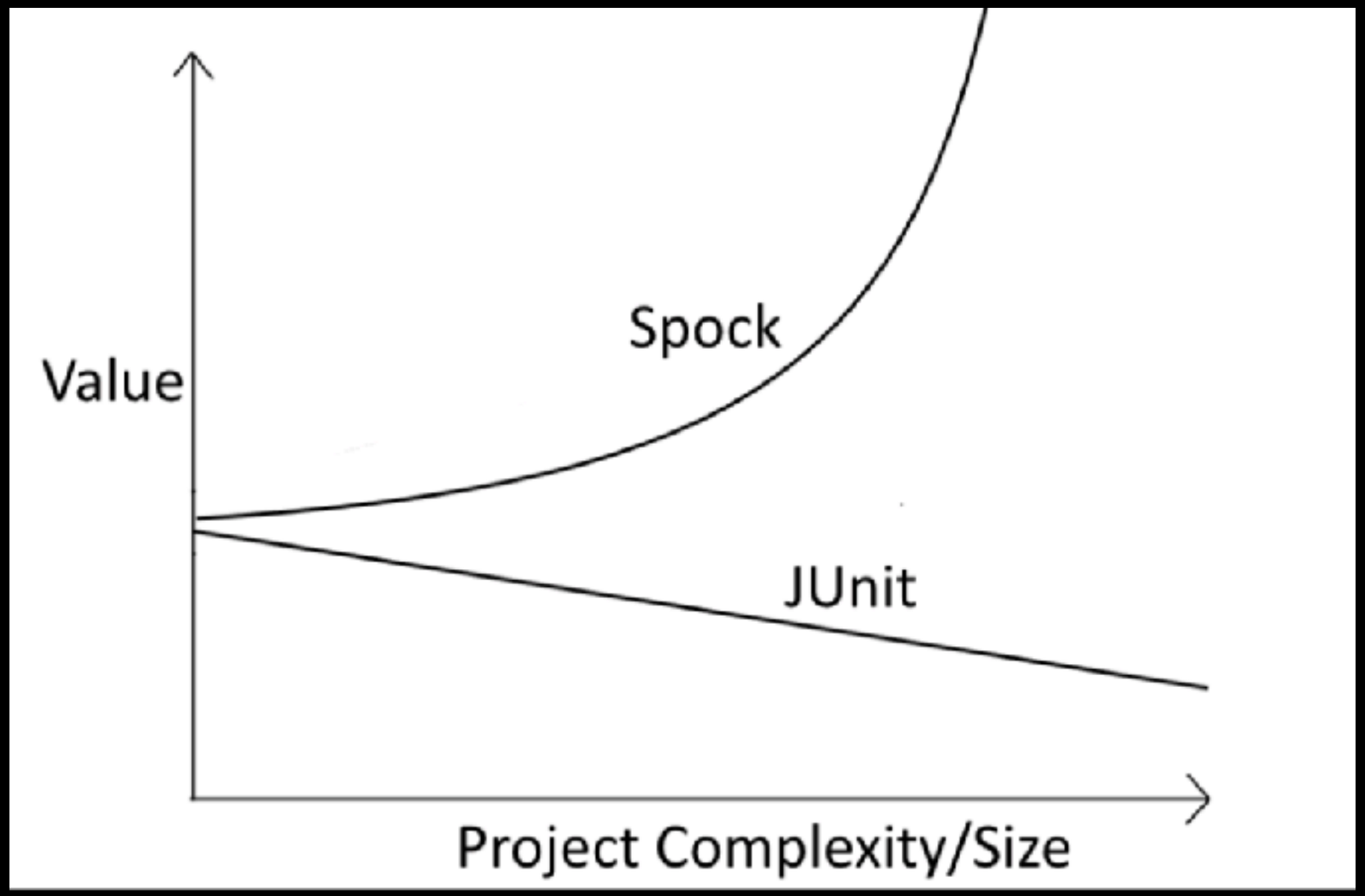
Why Spock



Spock for everything

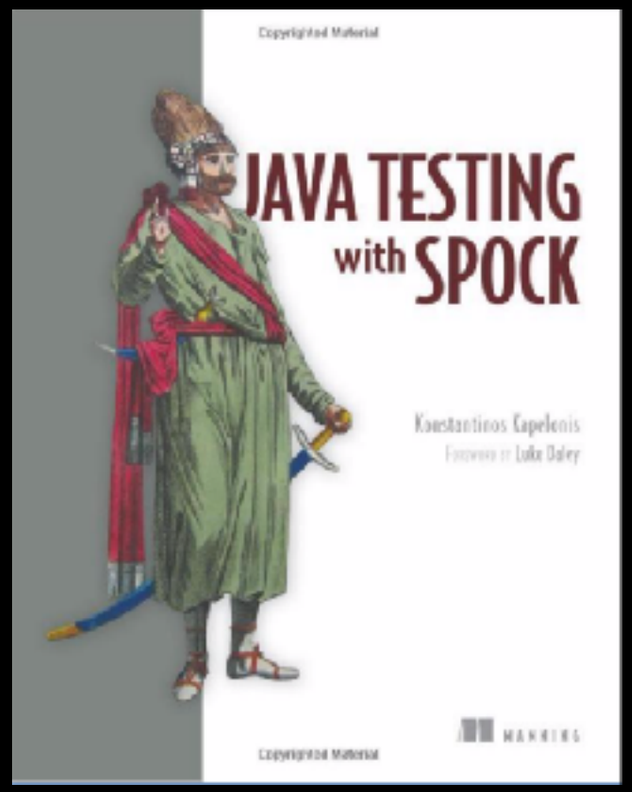


Why Spock(parameterized tests)

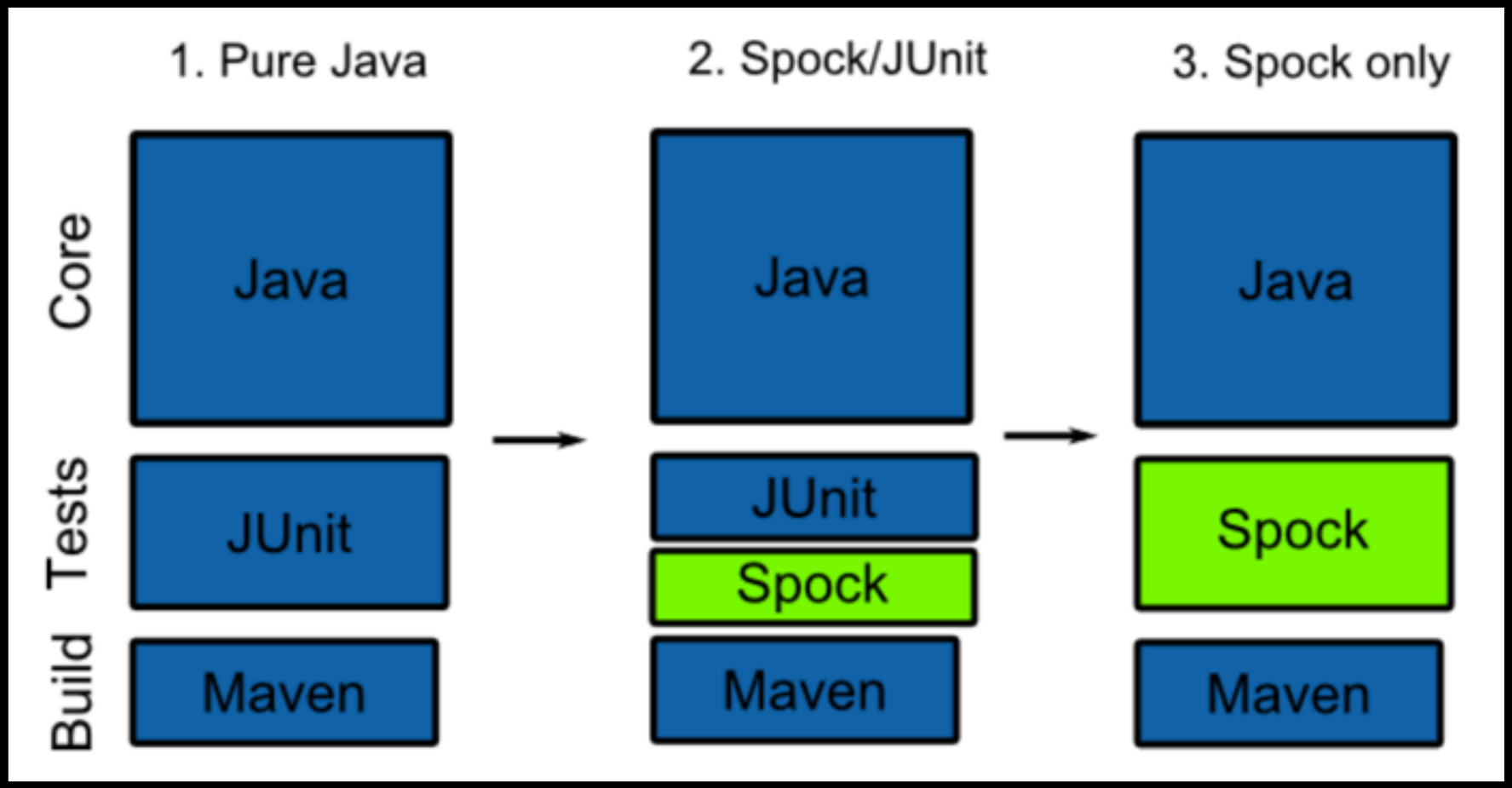


Spock With Java

- Spock uses JUnit Runner
- Spock is the default Grails test framework
- Compatible with all existing JUnit tools
- Spock and JUnit can coexist and run together



Gradual Spock acceptance



Spock VS Junit5

Reasons Why Spock is better

1.Test structure

Spock enforces the setup-trigger-assert paradigm

BBD-like specification

- clear separation in 3 blocks with predefined responsibility
  - given - creation, initialization and stubbing
  - when - operation to test
  - then - assertion and interaction verification
- unified (procedures for) of test creation
- improved readability
- easier to get started with writing (good) test
  - especially for less experienced people

Test structure - Junit 5

```java
class SimpleCalculatorTest {

    @Test
    void shouldAddTwoNumbers() {
        //given
        Calculator calculator = new Calculator();
        //when
        int result = calculator.add(1, 2);
        //then
        assertEquals(3, result);
    }
}
```

- Just plain code comments
- Easy to forget if template is not used
- Can be easily lost/misplaced in refactoring

Test structure - Spock

```
class SimpleCalculatorSpec extends Specification {

    def "should add two numbers"() {
        given:
            Calculator calculator = new Calculator()
        when:
            int result = calculator.add(1, 2)
        then:
            result == 3
    }
}
```

- [given]/when/then(or expect) required to compile code
- Clearly marks phases
- Especially handy with predefined test template

Spock clearly marks phases

```
def "If one sensor is active the alarm should sound as a precaution"() {
        given: "that only one fire sensor is active"
        FireEarlyWarning fireEarlyWarning =new FireEarlyWarning()
        int triggeredSensors = 1

        when: "we ask the status of fire control"
        fireEarlyWarning.feedData(triggeredSensors)
        WarningStatus status = fireEarlyWarning.getCurrentStatus()

        then: "only the alarm should be triggered"
        status.alarmActive
        !status.fireDepartmentNotified
}
```

2.Test Readability

Spock tests read like English sentences

English sentences

```
def "If one sensor is active the alarm should sound
  as a precaution"() {
      given: "that only one fire sensor is active"
       [...code here...]
      when: "we ask the status of fire control"
       [...code here...]
      then: "only the alarm should be triggered"
      [...code here...]
  }
```

Junit 5 reports

FireSensorTest

✓	sensorsAreTriggered
✓	everythingIsOk
✓	oneSensorIsTriggered
✓	twoSensorsAreTriggered

Spock reports

FireSensorSpec

	If all sensors are inactive everything is ok
	If one sensor is active the alarm should sound as a precaution
	If more than one sensors are active then we have a fire

Spock native report

- Work with non-developers
  - Readable by Testers
  - Readable by Business Analysts

Summary:

Created on Sun Jan 25 23:39:45 EET 2015 by Kostis

Executed features	Failures	Errors	Skipped	Success rate
3	0	0	0	100.0%

Features:

If all sensors are inactive everything is ok

Given: that all fire sensors are off
When: we ask the status of fire control
Then: no alarm/notification should be triggered

If one sensor is active the alarm should sound as a precaution

Given: that only fire sensor is active
When: we ask the status of fire control
Then: only the alarm should be triggered

If more than one sensors are active then we have a fire

Given: that two fire sensors is active
When: we ask the status of fire control
Then: alarm is triggered and the fire department is notified

APPROVED

3.Failed tests

Spock knows the context of failed tests

Junit 5 knows only actual result

刘硕@快手-IMServer

Junit 5 knows only actual result

Failure Trace

```
java.lang.AssertionError: 4 times (2 plus 3) is 20 expected:<20> but was:<25>
 at com.manning.spock.MultiplierTest.combinedOperationsTest(MultiplierTest.java:22)
```

Spock knows the context

Spock knows the context

Failure Trace

```
Condition not satisfied:

multi.multiply(4, adder.add(2, 3)) == 20
|     |           |     |          |
|     25          |     5          false
|                 com.manning.spock.Adder@691a0e79
com.manning.spock.Multiplier@38d9e447
```

Both sides of assert are analyzed

Failure Trace

JUnit assert

```
java.lang.AssertionError: Expected same result expected:<2> but was:<5>
 at com.manning.spock.chapter2.NormalAssert.methods(NormalAssert.java:42)
```

```
Caught: Assertion failed:
```

Groovy assert

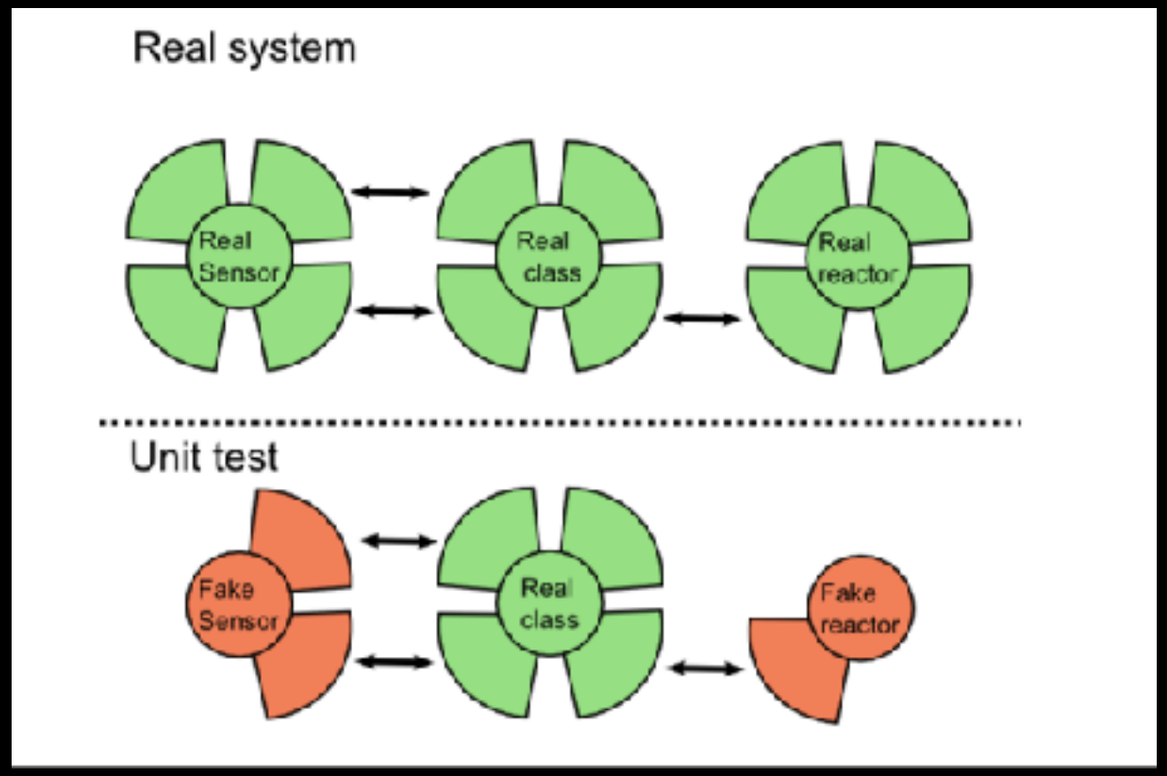
```
assert wordDetector.feedText(text).duplicatesFound().size() == 5
       |             |         |     |                   |      |
       |             |         |     [are, They]         2      false
       |             |         They are alone. They are a dying race.
       |             com.manning.spock.chapter2.WordDetector@552ee43b
       com.manning.spock.chapter2.WordDetector@552ee43b
```

4.Built-in mocking

Junit has no built-in mocking

Why we need mocking

- Testing with mocks(stubs) instead real Collaborators
- Stubbing call executions
- Verifying interactions



刘硕@快手-IMServer

Mocking - Junit5

```java
@Test
public void should_not_call_remote_service_if_found_in_cache() {
    //given
    given(cacheMock.getCachedOperator(CACHED_MOBILE_NUMBER)).willReturn(Optional.of(PLUS));
    //when
    service.checkOperator(CACHED_MOBILE_NUMBER);
    //then
    verify(wsMock, never()).checkOperator(CACHED_MOBILE_NUMBER);
}
```

- No built-in mocking framework
  - Mockito - first choice

Mocking - Spock

```
def "should not hit remote service if found in cache"() {
    given:
        cacheMock.getCachedOperator(CACHED_MOBILE_NUMBER) >> Optional.of(PLUS)
    when:
        service.checkOperator(CACHED_MOBILE_NUMBER)
    then:
        0 * wsMock.checkOperator(CACHED_MOBILE_NUMBER)
}
```

- Built-in mocking subsystem
- Extra short and more meaningful syntax
    - Thanks to Groovy operator overloading&AST transformations
    - Unbeatable by anything written in pure Java

Simple stubbing

```
given: “ a shopping basket”
Basket basket = new Basket()
and:"an empty warehouse"
WarehouseInventory inventory =
Stub(WarehouseInventory)
inventory.isEmpty() >> true
basket.setWarehouseInventory(inventory)
```

inventory.isEmpty() >> true

“When the method isEmpty() is called, ignore the real object and return true”

Complicated stubbing

given: "a basket, a TV and a camera"
Product tv = new Product(name:"bravia",price:1200,weight:18)
Product camera = new Product(name:"panasonic",price:
350,weight:2)
Basket basket = new Basket()
and:"a warehouse with partial availability"
WarehouseInventory inventory = Stub(WarehouseInventory{
isProductAvailable("bravia",1) >> true
isProductAvailable("panasonic",1) >> false
isEmpty() >> false
}

isProductAvailable("bravia",1)
>> true

“When the method isProductAvailable() is called with these arguments, return true”

Argument Matchers

```
WarehouseInventory inventory =
Stub(WarehouseInventory)
inventory.isProductAvailable( _, 1) >> true
basket.setWarehouseInventory(inventory)
```

(Mockito does not support partial matchers)

isProductAvailable(_,1) >> true

“When the method isProductAvailable() is called with any first argument and 1 as second argument then return true”

Method call count

```
and:"a warehouse with fluctuating stock levels"
WarehouseInventory inventory =
Stub(WarehouseInventory)
inventory.isProductAvailable( "bravia", _) >>>
true >> false
inventory.isEmpty() >>> [false, true]
basket.setWarehouseInventory(inventory)
```

inventory.isEmpty() >>> [false, true]

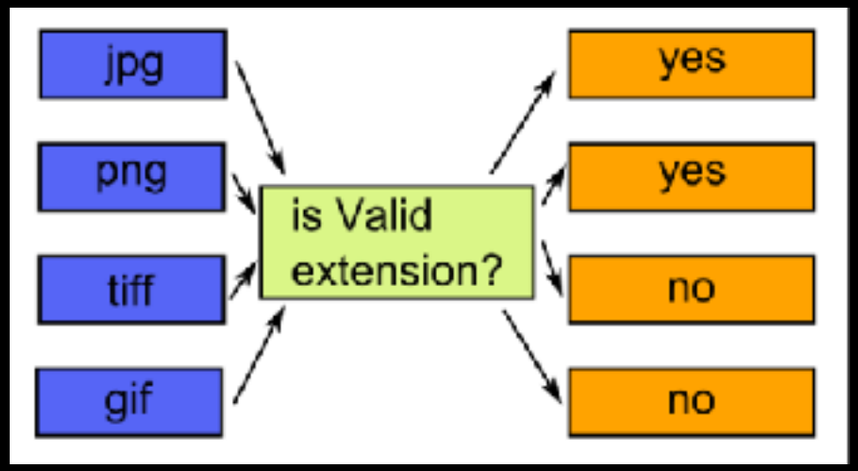
“When the method isEmpty() is called the first time return false. The second time it is called return true”

5.Parameterized tests

Common in big enterprise applications

Understanding parameterized tests

- One test for various input data
- Reduce code duplication
- Can hide specific business use cases



刘硕@快手-IMServer

Parameterized test - Junit5

- Input parameters rendered in report with @ParameterizedTest
- Complex declaration of custom data provider (method source)

```
@ParameterizedTest(name = "summing {0} and {1} should give {2}")
@MethodSource("integersProvider")
void shouldSumTwoIntegers(int x, int y, int expectedResult) {
    //when
    int result = calculator.add(x, y);
    //expect
    assertEquals(expectedResult, result);
}

private static Stream<Arguments> integersProvider() {
    return Stream.of(
            Arguments.of(1, 2, 3),
            Arguments.of(-2, 3, 1),
            Arguments.of(-1, -2, -3)
    );
}
```

刘牧@快手 IMSERVER

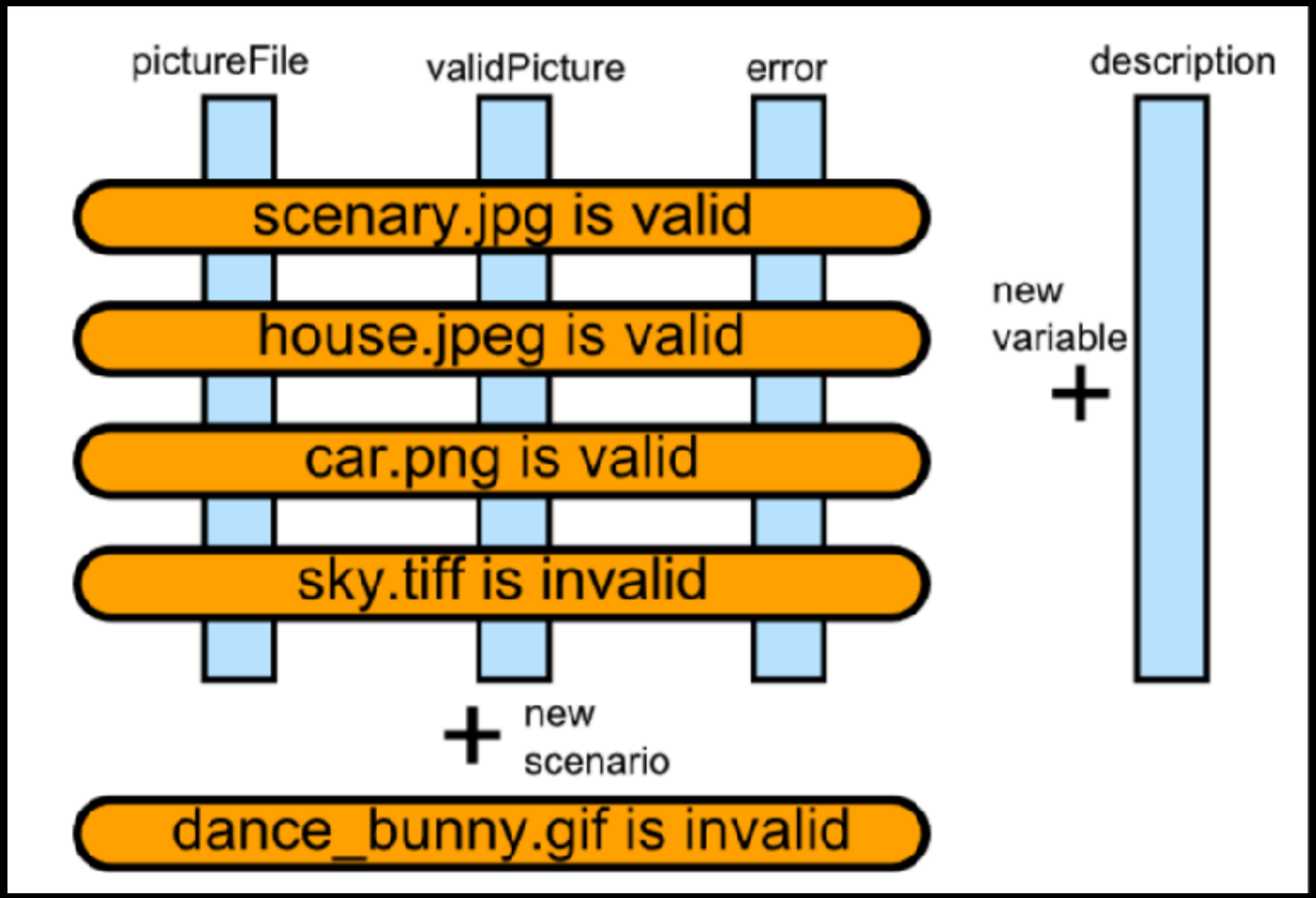
Parameterized test - Spock

- Tabular design
  - *where* keyword with table-like formatting
  - Very readable and easy to read
- Variables added implicitly(with proper type inferred by IDE)
- Input parameters possible to use in test name directly (with *#var* syntax)
- State of the art

```
@Unroll
def "should sum two integers (#x + #y = #expectedResult)"() {
    when:
        int result = calculator.add(x, y)
    then:
        result == expectedResult
    where:
         x |  y || expectedResult
         1 |  2 ||  3
        -2 |  3 ||  1
        -1 | -2 || -3
}
```

刘欣@快手-IMServer

Tabular design



Convert Specs directly into code

```
where: "possible nuclear incidents are:"
pressure | fireSensors | radiation              || alarm | shutDown | evacuation
150      | 0           | []                     || false | false    | -1
150      | 1           | []                     || true  | false    | -1
150      | 3           | []                     || true  | true     | -1
150      | 0           | [110.4f ,0.3f, 0.0f]   || true  | true     | 1
150      | 0           | [45.3f ,10.3f, 47.7f]  || false | false    | -1
155      | 0           | [0.0f ,0.0f, 0.0f]     || true  | false    | -1
170      | 0           | [0.0f ,0.0f, 0.0f]     || true  | true     | 3
180      | 0           | [110.4f ,0.3f, 0.0f]   || true  | true     | 1
500      | 0           | [110.4f ,300f, 0.0f]   || true  | true     | 1
30       | 0           | [110.4f ,1000f, 0.0f]  || true  | true     | 1
155      | 4           | [0.0f ,0.0f, 0.0f]     || true  | true     | -1
170      | 1           | [45.3f ,10.3f, 47.7f]  || true  | true     | 3
```

Junit and Spock LOC (same test)

```java
@RunWith(Parameterized.class)
public class NuclearReactorTest {
        private final int triggeredFireSensors;
        private final List<Float> radiationDataReadings;
        private final int pressure;

        private final boolean expectedAlarmStatus;
        private final boolean expectedShutdownCommand;
        private final int expectedMinutesToEvacuate;

        public NuclearReactorTest(int pressure, int triggeredFireSensors,
                        List<Float> radiationDataReadings, boolean expectedAlarmStatus,
                        boolean expectedShutdownCommand, int expectedMinutesToEvacuate) {

                this.triggeredFireSensors = triggeredFireSensors;
                this.radiationDataReadings = radiationDataReadings;
                this.pressure = pressure;
                this.expectedAlarmStatus = expectedAlarmStatus;
                this.expectedShutdownCommand = expectedShutdownCommand;
                this.expectedMinutesToEvacuate = expectedMinutesToEvacuate;

        }

        @Test
        public void nuclearReactorScenario() {
                NuclearReactorMonitor nuclearReactorMonitor = new NuclearReactorMonitor();

                nuclearReactorMonitor.feedFireSensorData(triggeredFireSensors);
                nuclearReactorMonitor.feedRadiationSensorData(radiationDataReadings);
                nuclearReactorMonitor.feedPressureInBar(pressure);
                NuclearReactorStatus status = nuclearReactorMonitor.getCurrentStatus();

                assertEquals("Expected no alarm", expectedAlarmStatus,
                                status.isAlarmActive());
                assertEquals("No notification", expectedShutdownCommand,
                                status.isShutdownNeeded());
                assertEquals("No notification", expectedMinutesToEvacuate,
                                status.getEvacuationMinutes());
        }

        @Parameters
        public static Collection<Object[]> data() {
                return Arrays
                                .asList(new Object[][] {
                                        { 150, 0, new ArrayList<Float>(), false, false, -1 },
                                        { 150, 1, new ArrayList<Float>(), true, false, -1 },
                                        { 150, 3, new ArrayList<Float>(), true, true, -1 },
                                        { 150, 0, Arrays.asList(110.4f, 0.3f, 0.0f), true,
                                                true, 1 },
                                        { 150, 0, Arrays.asList(45.3f, 10.3f, 47.7f), false,
                                                false, -1 },
                                        { 155, 0, Arrays.asList(0.0f, 0.0f, 0.0f), true, false,
                                                -1 },
                                        { 170, 0, Arrays.asList(0.0f, 0.0f, 0.0f), true, true,
                                                3 },
                                        { 180, 0, Arrays.asList(110.4f, 0.3f, 0.0f), true,
                                                true, 1 },
                                        { 500, 0, Arrays.asList(110.4f, 300f, 0.0f), true,
                                                true, 1 },
                                        { 30, 0, Arrays.asList(110.4f, 1000f, 0.0f), true,
                                                true, 1 },
                                        { 155, 4, Arrays.asList(0.0f, 0.0f, 0.0f), true, true,
                                                -1 },
                                        { 170, 1, Arrays.asList(45.3f, 10.3f, 47.7f), true,
                                                true, 3 }, });

        }

}
```

```groovy
class NuclearReactorSpec extends spock.lang.Specification{

        def "Complete test of all nuclear scenarios"() {
                given: "a nuclear reactor and sensor data"
                NuclearReactorMonitor nuclearReactorMonitor =new NuclearReactorMonitor()

                when: "we examine the sensor data"
                nuclearReactorMonitor.feedFireSensorData(fireSensors)
                nuclearReactorMonitor.feedRadiationSensorData(radiation)
                nuclearReactorMonitor.feedPressureInBar(pressure)
                NuclearReactorStatus status = nuclearReactorMonitor.getCurrentStatus()

                then: "we act according to safety requirements"
                status.alarmActive == alarm
                status.shutDownNeeded == shutDown
                status.evacuationMinutes == evacuation

                where: "possible nuclear incidents are:"
                pressure | fireSensors | radiation            || alarm | shutDown | evacuation
                150      | 0           | []                    || false | false    | -1
                150      | 1           | []                    || true  | false    | -1
                150      | 3           | []                    || true  | true     | -1
                150      | 0           | [110.4f ,0.3f, 0.0f]  || true  | true     | 1
                150      | 0           | [45.3f ,10.3f, 47.7f] || false | false    | -1
                155      | 0           | [0.0f ,0.0f, 0.0f]    || true  | false    | -1
                170      | 0           | [0.0f ,0.0f, 0.0f]    || true  | true     | 3
                180      | 0           | [110.4f ,0.3f, 0.0f]  || true  | true     | 1
                500      | 0           | [110.4f ,300f, 0.0f]  || true  | true     | 1
                30       | 0           | [110.4f ,1000f, 0.0f] || true  | true     | 1
                155      | 4           | [0.0f ,0.0f, 0.0f]    || true  | true     | -1
                170      | 1           | [45.3f ,10.3f, 47.7f] || true  | true     | 3
        }

}
```

6.Extra Enterprise features

Spock is ready for the Enterprise

Classic scenariao

- Tests should run in order
- If login fails no need to continue

```
public class SampleTest {
    @Test
    void login()

    @Test
    void createOrder()

    @Test
    void viewOrder()
}
```

@快手-IMServer

Spock @Stepwise

Used on class. If a test fails all other methods are ignored

Using @Stepwise

```
@Stepwise
class SpringRestSpec extends Specification {

    def "Simple status checker"() {
        [...code here...]
    }

    def "Cleaning all products"() {
        [...code here...]
    }

    def "Creating a product"() {
        [...code here...]
    }
}
```

Using Stepwise

Problems @ Javadoc Declaration Search Console Progress JUnit

Finished after 2,811 seconds

Runs: 3/3 (2 skipped) Errors: 1 Failures: 0

com.manning.spock.SpringRestSpec [Runner: JUnit 4] (2,131 s)

Simple status checker (2,110 s)

Cleaning all products (0,000 s)

Creating a product (0,000 s)

With Stepwise annotation

Problems @ Javadoc Declaration Search Console Progress JUnit

Finished after 4,04 seconds

Runs: 3/3 Errors: 3 Failures: 0

com.manning.spock.SpringRestSpec [Runner: JUnit 4] (3,650 s)

Simple status checker (1,480 s)

Cleaning all products (1,020 s)

Creating a product (1,130 s)

Without Stepwise annotation

Junit 5 @Ignore

Very simple. On/Off switch to enable/disable tests

@IgnoreIf({ os.windows })

This test will run on Linux/Mac
but not Win

```
@IgnoreIf({ env.containsKey("SKIP_SPOCK_TESTS") })
```

This test will not run if this system variable is present

Spock @Ignore

Use any condition that returns a boolean

```
@IgnoreIf({ new CreditCardProcessor().online() })
```

This test will not run if a staging server is down

More Spock features

- Mocking/Interaction testing
- Lifecycle methods
- Timeouts
- Data pipes/ Data generators
- Exception catching
- Functional tests with Geb
- Documentation annotations
- SpyObjects
- Spock extensions

Features comparison

	JUnit 5	Spock
development:	very good	could be better (only one dev)
learning curve:	very good (similar to 4)	good (Groovy to grasp)
tool support:	very good (trending up)	good (weaker compile time checks)
test structure:	good	very good (BDD by default)
exception testing:	good	very good
conditional execution:	good (limited custom logic)	very good (custom logic)
mocking:	good (Mockito)	good (very compact)
parameterized tests:	good	**very good** (exceptional!)

Summary

- if you prefer
  - old good Java as the only language
  - stability and being mainstream
  - stronger compile time error checking - choose JUnit 5
- if you favor
  - simplicity and readability
  - power of Groovy under the hood
  - beautiful parameterized tests and exception testing - choose Spock

Thank You

—Spock, a *Star Trek* character